1. Problem Title & Link

Title: **LeetCode 735: Asteroid Collision**

Link: https://leetcode.com/problems/asteroid-collision/

2. Problem Statement (Short Summary)

You are given an array asteroids.

Each asteroid has:

- **Size** → absolute value
- **Direction** → sign

positive → moving right

negative → moving left

Rules:

1. When two asteroids collide:
   - The **smaller asteroid explodes**
2. If both are equal:
   - **Both explode**
3. Asteroids moving in the **same direction never collide**

Goal:

Return the **final state of asteroids after all collisions**.

3. Examples (Input → Output)

Example 1

Input

asteroids = [5,10,-5]

Output

[5,10]

Explanation

10 and -5 collide

10 survives

Example 2

Input

asteroids = [8,-8]

Output

[]

Explanation

Both asteroids explode.

Example 3

Input

asteroids = [10,2,-5]

Output

[10]

Explanation

2 and -5 collide → -5 survives

10 and -5 collide → 10 survives

Example 4

Input

asteroids = [-2,-1,1,2]

Output

[-2,-1,1,2]

Explanation

No collisions occur.

4. Constraints

$2 \le$ asteroids.length $\le 10^4$

$-1000 \le$ asteroids[i] ≤ 1000

asteroids[i] $\ne 0$

Observations:

- Collisions only occur when:

positive asteroid followed by negative asteroid

5. Core Concept (Pattern / Topic)

Stack / Simulation

Why stack?

Because the **latest asteroid moving right** is the first candidate for collision.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Identify Collision Condition

Collision occurs only if:

stack top > 0

current asteroid < 0

Meaning:

→ right moving asteroid meets ← left moving asteroid

Step 2 — Collision Cases

Let:

stack top = a

current asteroid = b

Case 1

|a| > |b|

Current asteroid explodes.

Case 2

|a| < |b|

Stack asteroid explodes.

Remove it and check next.

Case 3

|a| == |b|

Both explode.

Step 3 — Use Stack

- Push asteroids moving right
- Resolve collisions when encountering a left-moving asteroid

7. Visual / Intuition Diagram (ASCII)

Example

[10,2,-5]

Processing

Stack: [10]

Stack: [10,2]

Incoming asteroid = -5

Collision

2 vs -5 → 2 destroyed

Next

10 vs -5 → -5 destroyed

Final

[10]

8. Pseudocode (Language Independent)

```
create empty stack

for asteroid in asteroids:

   while stack not empty AND
       stack top > 0 AND asteroid < 0:

      if abs(top) < abs(asteroid):
         pop stack
         continue

      if abs(top) == abs(asteroid):
         pop stack
         asteroid destroyed
         break

      if abs(top) > abs(asteroid):
         asteroid destroyed
         break

   if asteroid still alive:
      push to stack

return stack as array
```

9. Code Implementation

Python

```
class Solution:
   def asteroidCollision(self, asteroids):

      stack = []

      for a in asteroids:
```

```
        while stack and stack[-1] > 0 and a < 0:

            if abs(stack[-1]) < abs(a):
                stack.pop()
                continue

            elif abs(stack[-1]) == abs(a):
                stack.pop()
                a = 0
                break

            else:
                a = 0
                break

        if a:
            stack.append(a)

    return stack
```

Java

```
class Solution {
  public int[] asteroidCollision(int[] asteroids) {

    Stack<Integer> stack = new Stack<>();

    for(int a : asteroids){

      boolean destroyed = false;

      while(!stack.isEmpty() && stack.peek() > 0 && a < 0){

        if(Math.abs(stack.peek()) < Math.abs(a)){
          stack.pop();
          continue;
        }
```

```
            else if(Math.abs(stack.peek()) == Math.abs(a)){
                stack.pop();
                destroyed = true;
                break;
            }

            else{
                destroyed = true;
                break;
            }
        }

        if(!destroyed){
            stack.push(a);
        }
    }

    int[] result = new int[stack.size()];

    for(int i = result.length-1; i >= 0; i--){
        result[i] = stack.pop();
    }

    return result;
  }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Each asteroid is pushed and popped at most once.

Space Complexity

O(n)

Stack may store all asteroids.

11. Common Mistakes / Edge Cases

1. Forgetting collision condition

stack top > 0 AND current < 0

2. Not handling equal size collision
3. Pushing destroyed asteroid
4. Infinite loop in collision check

12. Detailed Dry Run

Example

asteroids = [5,10,-5]

Step	Asteroid	Stack	Action
1	5	[5]	push
2	10	[5,10]	push
3	-5	[5,10]	collision

Collision:

10 > 5

-5 destroyed

Final stack

[5,10]

13. Common Use Cases (Real-Life / Interview)

Stack simulations appear in:

- Physics collision simulations
- Undo/redo systems
- Expression evaluation
- Traffic flow simulations

14. Common Traps (Important!)

1. Treating all opposite signs as collision
2. Missing chained collisions
3. Forgetting stack top comparison
4. Incorrect break conditions

15. Builds To (Related LeetCode Problems)

Stack progression

LC 20 Valid Parentheses

LC 735 Asteroid Collision

LC 739 Daily Temperatures

LC 84 Largest Rectangle in Histogram

LC 901 Online Stock Span

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Simulation without stack	$O(n^2)$	inefficient
Stack simulation	$O(n)$	optimal

17. Why This Solution Works

The stack keeps track of **asteroids moving right that haven't collided yet**.

When a left-moving asteroid appears, we resolve collisions with the **most recent right-moving asteroid first**, which is exactly what a stack models.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Asteroids with **different speeds**
2. 2D asteroid collisions
3. Return **total destroyed asteroids**
4. Streaming asteroid simulation
5.